



UNIVERSITY OF RAJSHAHI

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

CSE4261 NEURAL NETWORKS AND DEEP LEARNING

Assignment-2

Author:

Name: MD. MIJU AHMED

ID: 2010676104

Supervisor:

SANGEETA BISWAS

ASSISTANT PROFESSOR

July 25, 2025

Contents

1	Analysis of Activation Functions	2
2	CNN Models and Their Kernel Types	2
3	Feature Map Evolution in InceptionResNetV2	3
3.1	Stem Block	3
3.2	Inception-ResNet-A Block (Repeated 5x)	3
3.3	Reduction-A Block	3
3.4	Inception-ResNet-B Block (Repeated 10x)	3
3.5	Reduction-B Block	4
3.6	Inception-ResNet-C Block (Repeated 5x)	4
3.7	Final Layers	4

1 Analysis of Activation Functions

The activation function used in the head of a neural network is critical for producing the correct output format for a given task. The following table shows the default activation function used in the body of various CNN architectures and compares the final classification accuracy when the head (final layer) uses either ReLU or Softmax.

Table 1: Model Accuracy with Different Head Activation Functions

Model	Activation Function Used in Body	Accuracy with ReLU in Head	Accuracy with Softmax in Head
MobileNetV2	ReLU6	15.95%	69.80%
ResNet50	ReLU	6.45%	5.65%
VGG16	ReLU	8.95%	25.80%
EfficientNetB0	Swish	5.00%	5.00%
DenseNet121	ReLU	9.20%	66.05%
NASNetMobile	ReLU	24.05%	72.85%
EfficientNetV2B0	Swish	5.00%	5.00%
InceptionV3	ReLU	28.60%	74.85%
Xception	ReLU	47.25%	76.70%
InceptionResNetV2	ReLU	28.50%	79.30%

Observation

As shown in the data, using ReLU as the final activation function for a classification task results in a significant decrease in accuracy for most models compared to using Softmax. This is because ReLU outputs a raw value, which is not suitable for multi-class probability distribution. Softmax, on the other hand, converts the network's outputs into a set of probabilities that sum to one, which is the correct format for classification, thereby achieving much higher performance.

2 CNN Models and Their Kernel Types

Modern CNNs employ a variety of convolutional kernel types to optimize for performance, efficiency, and adaptability. Different kernel designs are suited for different tasks. The table below lists several kernel types and some of the CNN models that utilize them.

Table 2: Examples of CNN Models for Different Kernel Types

Kernel Type	Associated CNN Models
Regular kernel	VGG16, ResNet50, DenseNet121
Deformable kernel	Deformable ConvNet v1, Deformable ConvNet v2, YOLOv4
Dilated kernel	DeepLabv3, WaveNet, ESPNet
Depthwise separable kernel	MobileNetV1, Xception, EfficientNet
Modified Depthwise-Separable kernel	MobileNetV2, MobileNetV3, FBNet
Pointwise kernel	InceptionV3, ResNet (Bottleneck), MobileNet series

3 Feature Map Evolution in InceptionResNetV2

The chosen CNN model for this analysis is **InceptionResNetV2**. Below is a layer-by-layer breakdown of the architecture, focusing on how feature maps are transformed from low-level patterns to high-level object representations.

3.1 Stem Block

The initial block processes the input image to create a base set of low-level features.

- **Input:** $224 \times 224 \times 3$ (A standard ImageNet input size).
- **Layers & Output Dimensions:**
 - 3×3 Conv $\rightarrow$ 32 filters $\rightarrow 149 \times 149 \times 32$
 - 3×3 Conv $\rightarrow$ 32 filters $\rightarrow 147 \times 147 \times 32$
 - 3×3 Conv $\rightarrow$ 64 filters $\rightarrow 147 \times 147 \times 64$
 - MaxPool + Conv $\rightarrow 73 \times 73 \times 192$
- **Feature Maps:** The feature maps at this stage **capture edges, gradients, and color blobs**. These are fundamental, low-level features.

3.2 Inception-ResNet-A Block (Repeated 5x)

This block begins to form more complex patterns from the basic features.

- **Output:** $35 \times 35 \times 320$.
- **Structure:** Each block contains parallel branches with 1×1 , 3×3 , and 5×5 convolutions. The outputs are concatenated and combined with a residual shortcut connection.
- **Feature Maps:** These maps learn **mid-level patterns such as corners, textures, and basic shapes**.

3.3 Reduction-A Block

This block downsamples the feature maps to reduce spatial dimensions and increase channel depth, forcing the network to learn more abstract features.

- **Output:** Downsamples to $17 \times 17 \times 1088$.
- **Feature Maps:** The network now begins learning identifiable **object parts like heads, wheels, and leaves**.

3.4 Inception-ResNet-B Block (Repeated 10x)

This block uses wider, asymmetrical filters to capture more elongated and complex shapes.

- **Output:** $17 \times 17 \times 1088$.
- **Structure:** Employs narrower filters, such as 1×7 and 7×1 , to effectively capture asymmetrical patterns.
- **Feature Maps:** The maps now represent larger, more composite structures like **faces, windows, and animal parts**.

3.5 Reduction-B Block

Further downsampling prepares the feature maps for final object-level recognition.

- **Output:** $8 \times 8 \times 2080$.
- **Feature Maps:** At this stage, the feature maps are abstract enough to **represent full objects such as cars and dogs**.

3.6 Inception-ResNet-C Block (Repeated 5x)

This is the final block for refining complex object features before classification.

- **Output:** $8 \times 8 \times 1536$.
- **Feature Maps:** These maps represent the final, polished **object-level concepts** right before the classification head takes over.

3.7 Final Layers

The final layers condense all the spatial information into a probability distribution for classification.

- **Structure:**
 - Global Average Pooling $\rightarrow 1 \times 1 \times 1536$
 - Dropout + Dense Layer $\rightarrow$ Softmax output
- **Feature Maps:** At this point, the feature map is fully condensed, representing a complete **object understanding, with one feature per class** ready for the final prediction.

Assignment Github Link : Assignment